

AIRSPACE INTELLIGENCE

Real-Time Aircraft Monitoring over the Iberian Peninsula

Replication Guide & Project Report

Group 4 Members: Rincon Juan José, Gelin Romain, Gaitan Diego, Tcheishvili Luka, Tambey Cécile

Field	Details
Course	Modern Data Architectures for Big Data II
Institution	IE University — School of Science & Technology
Pipeline	OpenSky API → Apache NiFi → MinIO (Bronze) → Apache Spark → Jupyter
Spark Capabilities	Spark Structured Streaming, Spark ML (KMeans), Spark SQL
Data Region	Iberian Peninsula (lat 35.5–44.5, lon -10.0–4.5)
Deadline	March 19th, 2026
Submission	Blackboard (PDF slide deck + Notebook + MP4/MOV recordings)

1. Project Overview

Airspace Intelligence is a real-time Big Data pipeline that continuously ingests live aircraft telemetry from the OpenSky Network API, stores raw snapshots in MinIO object storage, and performs three distinct analytical use cases using Apache Spark. The monitored region is the Iberian Peninsula, bounded by latitudes 35.5 to 44.5 degrees North and longitudes -10.0 to 4.5 degrees East.

1.1 Architecture Summary

Layer	Technology & Role
Ingestion	Apache NiFi — polls OpenSky API every 10 seconds, writes JSON snapshots to MinIO
Storage (Bronze)	MinIO — S3-compatible object store, bucket: airspace-intelligence-bronze
Processing	Apache Spark (PySpark) with hadoop-aws + aws-java-sdk-bundle JARs
Analysis 1	Spark Structured Streaming — real-time density monitoring with tumbling windows
Analysis 2	Spark SQL — traffic by country and airline, market share, cruising speeds
Analysis 3	Spark ML KMeans — congestion detection and geographic cluster labeling
Visualisation	Seaborn + Matplotlib (dark theme) + Folium interactive map

2. Access Credentials

CONFIDENTIAL — For team members and professor only. Do not share publicly.

Credential	Value
MinIO Access Key	airspace-intel
MinIO Secret Key	Confidential
MinIO S3 Endpoint	http://127.0.0.1:9000
MinIO Web Console	http://127.0.0.1:9001
Bronze Bucket Name	airspace-intelligence-bronze
Bronze Path (Spark)	s3a://airspace-intelligence-bronze/opensky/bronze/
OpenSky API URL	https://opensky-network.org/api/states/all
Bounding Box	lamin=35.5 lomin=-10.0 lamax=44.5 lomax=4.5

3. MinIO Setup

MinIO acts as our Bronze storage layer — an S3-compatible object store that holds all raw JSON snapshots ingested by NiFi. Follow these steps exactly.

3.1 Start the MinIO Server

Open a terminal and run the following command. MinIO must be running before starting NiFi.

```
minio server --address :9000 --console-address :9001 /data/s3
```

Parameter	Explanation
--address :9000	S3 API port — used by NiFi (PutS3Object) and Spark (s3a://) to read/write files
--console-address :9001	Web UI port — open in browser to manage buckets and browse files visually
/data/s3	Physical directory on disk where all bucket data is stored

3.2 Verify MinIO is Running

1. Open a browser and go to: <http://127.0.0.1:9001>
2. Log in using the access credentials in Section 2
3. You should see the MinIO dashboard
4. Run in terminal to confirm port is listening:

```
sudo ss -tulnp | grep 9000
```

3.3 Create the Bronze Bucket

5. In the MinIO web console, click Buckets in the left sidebar
6. Click Create Bucket
7. Enter the bucket name exactly: `airspace-intelligence-bronze`
8. Leave all other settings as default — Access will show as PRIVATE (correct)
9. Click Create Bucket to confirm

The bucket name must match exactly. Spark and NiFi both reference this name. Access will be shown as PRIVATE — this is expected and correct.

Once NiFi is running, the bucket will show the following details (matching what was observed in this project):

Field	Expected Value
Bucket Name	airspace-intelligence-bronze
Access	PRIVATE

File path inside bucket	opensky / bronze / YYYY-MM-DD / HH / opensky_YYYYMMDD_HHmmss_SSS.json
File size per snapshot	~49.2 to 50.0 KiB
Total size after 67 snapshots	approximately 3.1 MiB
Total objects after ~10 min	67 objects

3.4 Validate Data is Arriving

After NiFi is running (Section 4), return to the MinIO console to verify files are being written.

10. In MinIO console, click on airspace-intelligence-bronze
11. Navigate to: opensky / bronze / 2026-03-12 / 18 / (date and hour will match your run time)
12. You should see .json files named like: opensky_20260312_183023_105.json
13. Each file should be approximately 49-50 KiB

Files are stored in dated subfolders (year/month/day/hour). This is why Spark requires recursiveFileLookup=true when reading — a flat path glob will find nothing.

4. Apache NiFi — Ingestion Pipeline

NiFi is responsible for polling the OpenSky API every 10 seconds and writing each response as a JSON file to MinIO. The pipeline consists of three processors connected in sequence.

NiFi must be started AFTER MinIO is running. The pipeline flow is: GenerateFlowFile -> InvokeHTTP -> PutS3Object

4.1 Create a Process Group

14. Open the NiFi UI (typically <http://127.0.0.1:8080/nifi>)
15. Drag a Process Group onto the canvas from the toolbar
16. Name it: airspace-intelligence
17. Double-click the Process Group to enter it

4.2 Processor 1 — GenerateFlowFile

This processor triggers the pipeline on a schedule. It creates an empty FlowFile every 10 seconds to kick off the API call.

18. Drag a Processor onto the canvas, search for GenerateFlowFile, click Add
19. Right-click the processor, click Configure
20. Go to the SCHEDULING tab:
 - Set Run Schedule to: 10 sec
21. Go to the PROPERTIES tab:
 - Set File Size to: 0 b
 - Set Batch Size to: 1
22. Click APPLY

4.3 Processor 2 — InvokeHTTP

This processor makes the actual HTTP GET request to the OpenSky Network API and returns the JSON response as the FlowFile content.

23. Drag a Processor onto the canvas, search for InvokeHTTP, click Add
24. Right-click the processor, click Configure
25. Go to the PROPERTIES tab and set the following:

Property	Value to Set
HTTP Method	GET

Remote URL	https://opensky-network.org/api/states/all?lamin=35.5&lomin=-10.0&lamax=44.5&lomax=4.5
Connection Timeout	10 sec
Read Timeout	30 sec
Include Date Header	True
Follow Redirects	True

26. Go to the RELATIONSHIPS tab:

- Auto-terminate: Failure, No Retry, Original, Retry
- Only the Response relationship should flow to the next processor

27. Click APPLY

4.4 Processor 3 — LogAttribute

This processor logs FlowFile attributes for debugging and monitoring. It sits between InvokeHTTP and PutS3Object so you can inspect what NiFi received from the API before writing to MinIO.

28. Drag a Processor onto the canvas, search for LogAttribute, click Add

29. Right-click the processor, click Configure

30. Go to the PROPERTIES tab:

- Log Level: info
- Log Payload: false (we only need attribute metadata, not the full JSON body)

31. Go to the RELATIONSHIPS tab: Auto-terminate nothing — connect success to PutS3Object

32. Click APPLY

4.5 Processor 4 — PutS3Object

This processor writes the FlowFile (the JSON response from OpenSky) to the MinIO Bronze bucket. It uses the S3 API protocol, which MinIO fully supports.

33. Drag a Processor onto the canvas, search for PutS3Object, click Add

34. Right-click the processor, click Configure

35. Go to the PROPERTIES tab and set the following exactly:

Property	Value to Set
Bucket	airspace-intelligence-bronze
Object Key	opensky/bronze/\${now():format('yyyy-MM-dd')}/\${now():format('HH')}/opensky_\${now():format('yyyyMMdd_HH:mm:ss_SSS')}.json
Region	us-east-1

Access Key ID	airspace-intel
Secret Access Key	Confidential
Endpoint Override URL	http://127.0.0.1:9000
Use Path Style Access	True
SSL Context Service	(leave empty — we are using HTTP not HTTPS)
Storage Class	STANDARD

36. Go to the RELATIONSHIPS tab:

- Auto-terminate: Failure, Success

37. Click APPLY

The Object Key expression uses NiFi's Expression Language. It dynamically names each file with the current date and timestamp, creating a folder hierarchy by date and hour automatically.

4.6 Connect the Processors

Connect all four processors in sequence. The full flow is: GenerateFlowFile -> InvokeHTTP -> LogAttribute -> PutS3Object

38. Hover over GenerateFlowFile, drag arrow to InvokeHTTP — select success relationship, click ADD

39. Hover over InvokeHTTP, drag arrow to LogAttribute — select Response relationship, click ADD

40. Hover over LogAttribute, drag arrow to PutS3Object — select success relationship, click ADD

Auto-terminate all unused relationships on each processor (Failure, No Retry, Original, Retry on InvokeHTTP; Failure on PutS3Object).

4.7 Start the Pipeline

41. Right-click on empty canvas space, click Select All

42. Click the Start button (triangle icon) in the toolbar

43. All three processors should turn green

44. Watch the statistics counters on each processor — they should show data flowing every 10 seconds

45. After 60 seconds, check MinIO console to confirm files are appearing

If PutS3Object shows errors, check that MinIO is running on port 9000 and the bucket name is exactly airspace-intelligence-bronze (all lowercase, hyphens).

5. Jupyter Notebook — Spark Analysis

The notebook connects to MinIO via the S3A protocol, loads all Bronze JSON snapshots into a Spark DataFrame, and runs three use cases. Below is a summary of each section and what it does.

5.1 Environment Setup (Section 0)

- Installs required Python packages: pyspark, boto3, folium, seaborn, matplotlib
- Imports all libraries
- Sets Matplotlib to dark_background theme and Seaborn to darkgrid style

5.2 Spark Session (Section 1)

Creates a SparkSession with the S3A connector JARs. The key configuration points are:

- hadoop-aws:3.3.4 and aws-java-sdk-bundle:1.12.262 loaded as packages
- fs.s3a.endpoint set to http://127.0.0.1:9000 (MinIO)
- fs.s3a.path.style.access set to true (required for MinIO)
- fs.s3a.connection.ssl.enabled set to false (HTTP not HTTPS)

5.3 Data Load (Section 2)

Reads all JSON files from the Bronze bucket into a Spark DataFrame. Critical options:

- Schema is manually defined: {time: LongType, states: ArrayType(ArrayType(StringType))}
- multiLine: true — each file is a single JSON object
- recursiveFileLookup: true — files are nested in date/hour subfolders

After loading, the states array is exploded and each position in the array is mapped to a named column: icao24, callsign, origin_country, longitude, latitude, baro_altitude, on_ground, velocity, vertical_rate. Filters remove ground aircraft and invalid coordinates. The result is cached.

5.4 Use Case 1 — Airspace Density Monitoring (Section 3)

Answers: How many aircraft are in the monitored airspace at any given moment, and how does this change over time?

- Static analysis: counts distinct aircraft per snapshot timestamp using approx_count_distinct
- Structured Streaming: reads Bronze path as a stream, applies a 1-minute tumbling window, sinks to memory table density_stream, runs for 25 seconds
- Hourly aggregation: groups by hour, counts unique aircraft, computes average velocity and altitude
- Charts produced: time-series line chart of aircraft count per snapshot, bar chart of unique aircraft by hour

5.5 Use Case 2 — Traffic by Country and Airline (Section 4)

Answers: Which countries and airlines dominate the Iberian airspace, and how fast are they flying?

- Spark SQL is used throughout — aircraft_df registered as a temporary view
- Top 20 countries ranked by unique aircraft count
- Airline code extracted from callsign using regex (first 3 uppercase alpha characters)
- Market share computed as percentage of total unique aircraft per country
- Average cruising speed converted from m/s to km/h, filtered to countries with at least 3 aircraft
- Charts: horizontal bar (countries), horizontal bar (airlines), pie chart (market share), bar chart (speeds)

5.6 Use Case 3 — Congestion Detection with KMeans (Section 5)

Answers: Which geographic grid cells are consistently congested, and can we cluster them by density pattern?

- Airspace divided into 2x2 degree grid cells
- A cell is marked congested when it contains 5 or more aircraft in a snapshot
- KMeans clustering (k=3, seed=42) applied on [lat_cell, lon_cell, avg_aircraft]
- Silhouette score computed to evaluate cluster quality
- Clusters labeled Low/Medium/High Density based on mean aircraft count per cluster
- Charts: heatmap of average aircraft per cell, congestion percentage over time, KMeans bubble scatter plot, Folium interactive dark map with color-coded congestion rectangles

6. Project Insights

All numbers below are from the executed notebook run on March 12, 2026 — 67 snapshots collected over approximately 10 minutes of NiFi ingestion.

6.1 Use Case 1 — Airspace Density

Metric	Value
Total aircraft records loaded	23,579
Valid snapshots analysed	60 out of 67 raw files
Average aircraft per snapshot	352.8 aircraft
Peak aircraft count	375 aircraft (18:29 UTC)
Unique aircraft observed (hour 18)	535 distinct ICAO24 codes
Average velocity (hourly)	183.7 m/s
Average altitude (hourly)	7,527 m (~24,700 ft)

- The airspace density remained remarkably stable across the 10-minute window, hovering between 350 and 375 aircraft. This consistency suggests the capture window fell within peak early-evening traffic, a known high-demand period for European routes.
- 535 unique ICAO24 codes appeared across the hour, significantly more than the 352 average per snapshot. This turnover indicates aircraft entering and leaving the bounding box continuously — the Iberian Peninsula is a high-throughput transit corridor.
- The streaming window (1-minute tumbling) returned an empty result in the demo run due to the short 25-second query duration — in a production deployment running continuously this would populate with real-time windowed counts.

6.2 Use Case 2 — Traffic by Country and Airline

Top 10 Countries by Unique Aircraft

Country	Unique A/C	Market Share	Avg Speed (km/h)	Avg Alt (m)
Spain	141	26.4%	569	5,404
Ireland	57	10.7%	779	9,548
France	52	9.7%	602	7,076
United Kingdom	39	7.3%	765	9,685
Portugal	38	7.1%	527	5,182
Malta	28	5.2%	741	8,825

Austria	27	5.0%	659	7,712
Germany	24	4.5%	699	8,669
Morocco	17	3.2%	821	10,705
Netherlands	14	2.6%	732	8,450

- Spain dominates with 26.4% market share (141 unique aircraft) — unsurprisingly, as the monitored region is centred on the Iberian Peninsula and Spanish domestic and short-haul routes are highly active in early evening.
- Morocco appears in 9th place at 821 km/h average speed and 10,705 m altitude — the highest of all countries. These are long-haul overflights at cruising altitude, not local traffic. Morocco-registered aircraft are transiting, not landing.
- Spain and Portugal fly significantly lower (5,404 m and 5,182 m) and slower (569 and 527 km/h) than all other countries — consistent with short-haul regional flights and aircraft on approach or departure phases.
- Ireland at 10.7% is striking — Irish-registered aircraft (predominantly Ryanair, registered in Dublin) are the second largest group, reflecting Ireland's role as the home jurisdiction for Europe's largest low-cost carrier.

Top 5 Airlines by Unique Aircraft

Airline Code	Full Name	Unique A/C	Observations	Avg Speed (km/h)
RYP	Ryanair	83	4,143	756
VLG	Vueling	47	1,930	649
EJU	easyJet Europe	27	1,345	738
TAP	TAP Air Portugal	21	662	671
ANE	Air Nostrum	21	943	569

- Ryanair (RYP) is the single largest operator with 83 unique aircraft — nearly double the next competitor. With 4,143 observations it also has the highest dwell time in the monitored region, confirming its dense Iberian route network.
- Royal Air Maroc (RAM) recorded the highest average speed of all airlines at 237.98 m/s (858 km/h) — well above typical cruising speed — suggesting these are long-distance flights captured at optimal cruise altitude with strong tailwinds.
- RVP (a cargo/private operator) recorded an average speed of only 46.53 m/s (167 km/h), which indicates turboprop or light aircraft operations fundamentally different from commercial jets.

6.3 Use Case 3 — Congestion Detection

Metric	Value
Grid resolution	2x2 degree cells

Congestion threshold	5+ aircraft per cell per snapshot
Total cell-snapshot observations	2,410
Congested observations	1,512 (62.7% of all)
KMeans silhouette score	0.4687 (moderate separation)
High Density cluster cells	4 cells, avg 28.5 aircraft
Medium Density cluster cells	15 cells, avg 11.0 aircraft
Low Density cluster cells	26 cells, avg 3.1 aircraft

Top 5 Congestion Hotspots (Persistent across all 60 snapshots)

Cell ID	Lat	Lon	Avg Aircraft	Peak Aircraft
LAT40_LON-4	40.0	-4.0	37.2	44
LAT38_LON-10	38.0	-10.0	26.4	29
LAT40_LON2	40.0	2.0	26.9	30
LAT36_LON-6	36.0	-6.0	23.4	27
LAT40_LON-2	40.0	-2.0 (UC)	6.8	9

- 62.7% of all cell-snapshot observations exceeded the congestion threshold of 5 aircraft. This is remarkably high and confirms the Iberian Peninsula sits at the intersection of multiple major European air corridors.
- The single most congested cell is LAT40_LON-4 (central Spain, near Madrid), averaging 37.2 aircraft per 2x2 degree cell and peaking at 44. This cell sits directly over the Madrid FIR (Flight Information Region), one of the busiest airspace zones in Europe.
- LAT38_LON-10 (southwest Portugal / Atlantic entry point) averages 26.4 aircraft — this cell captures aircraft entering/exiting European airspace from transatlantic routes, explaining the persistently high count.
- The KMeans silhouette score of 0.4687 indicates moderate but meaningful cluster separation. The three clusters cleanly separate strategic chokepoints (High), busy transit corridors (Medium), and peripheral airspace (Low).
- All 30 congested hotspot cells were congested in every single snapshot (60/60 observations), meaning congestion in Iberian airspace is not random or transient — it is structural and predictable.

7. Conclusions

- The pipeline successfully demonstrates a complete real-time Big Data architecture: live API ingestion via NiFi, raw storage in MinIO, and multi-modal Spark analysis in a single notebook.
- The Iberian Peninsula is a structurally congested airspace. 62.7% congestion rate across all grid cells confirms this is not time-of-day noise but a baseline characteristic of the region.
- Spain and Ireland together account for 37% of all unique aircraft, and Ryanair alone accounts for roughly 15% of all unique aircraft — low-cost carrier dominance is clearly visible in raw telemetry data.
- Morocco-registered and long-haul aircraft transit at significantly higher altitudes and speeds than domestic/regional operators, creating a natural two-tier structure in the airspace that KMeans captures.
- Spark Structured Streaming, Spark SQL, and Spark ML KMeans were each applied to distinct analytical questions, satisfying the course requirement for multiple advanced Spark capabilities.

7.1 Potential Future Work

- Extend NiFi ingestion to 24+ hours to capture full daily traffic patterns and identify morning vs evening peaks
- Add a Silver layer transformation in Spark to enrich data with airport proximity (nearest airport lookup)
- Implement a Gold layer serving technology such as Apache Cassandra or PostgreSQL to persist aggregated insights
- Use GraphFrames (Spark Graph Processing) to model airport-to-airport routes as a directed graph and identify hub connectivity
- Deploy a real-time dashboard using Apache Superset connected to the Gold layer for live monitoring